



MEMORY-AWARE SOFTWARE ENGINEER- ING AGENTS

AI4SE @ KI 2026

Architectures, Mechanisms, and Open Gaps



Overview

The Memory Problem

Production Landscape

Memory Architectures

SE-Specific Mechanisms

Open Gaps & Vision



01

The Memory Problem



Software Engineering Agents Are Stateless

- Agentic AI now operates across the **full SE lifecycle**: bug resolution, code review, architectural analysis
- Each session accumulates significant task-specific knowledge
- Yet that knowledge **vanishes when the session ends**
- The next invocation starts from scratch — repeating investigations, rediscovering constraints, re-learning conventions

The Amnesia Problem

Every major AI provider now ships persistent memory.
Yet fundamental capabilities — episodic recall, temporal versioning, decision provenance — remain absent.

The CoALA Memory Taxonomy

The CoALA framework maps human memory types onto agent components:

- Working memory — active context window
- Semantic memory — facts, conventions
- Episodic memory — past experiences
- Procedural memory — learned workflows

Current state in production:

- ✓ Working: well-served by context windows
- ~ Semantic: partially via instruction files
- × **Episodic: almost entirely absent**
- × **Procedural: the rarest type**

Sumers et al., “Cognitive Architectures for Language Agents” (TMLR, 2024)



02

Production Landscape



Ten SE Harnesses, Twenty Dimensions

- Feature-level analysis of **10 production SE agent harnesses**
- 20 dimensions across six categories:
 - CoALA memory types
 - Memory operations
 - Temporal mechanisms
 - SE-specific capabilities
 - Collaboration & governance
 - Integration

Central Finding: The Episodic-Temporal Deficit

No production harness ships episodic memory or temporal versioning natively. The capabilities most critical for learning agents are those no tool implements.



Harness Memory Capabilities

	 Claude Code	 Codex CLI	 Cursor	 Copilot	 Windsurf	 Gemini CLI	 OpenCode	 Devin	 Aider	 Open-Hands
Human-written instructions	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Auto-generated memory	✓	✓	✗	✓	✓	✓	●	●	✗	✗
Context compaction	✓	✓	✓	✓	●	✓	✓	●	✗	✓
Team-shared memory	●	●	●	✓	✗	●	●	✓	●	✗
Temporal versioning	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Episodic memory	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

✓ Present ● Partial ✗ Absent

All 10 harnesses support instruction files (semantic memory).
Five generate memories automatically — but none distinguishes factual from experiential.

Temporal versioning and episodic memory: universally absent.



The Community Response: MCP Memory Add-Ons

- The gap has spawned **40+ MCP-based memory servers**
- Nine of ten major coding agents now support MCP — making add-ons agent-agnostic
- Key examples:
 - claude-mem — lifecycle-hook session capture
 - AgentMemory — hybrid BM25 + vector + knowledge graph
 - Graphiti/Zep — **only system with bi-temporal validity**
 - Codebase-Memory — Tree-Sitter knowledge graph
 - Hindsight — failed attempt replay
- Stark imbalance: semantic near-universal, temporal/procedural nearly absent



The Full Ecosystem

	Memory Types			Memory Operations				Temporal			Software Engineering					Collaboration & Governance			Integration	
	Semantic	Episodic	Procedural	Auto-Generated	Compaction	Fact./Experiential	Staleness	Temporal Metadata	Temporal Versioning	Bi-Temporal	Code-Structural	Cross-Session	Provenance	Failed Replay	Git-Aware	Team/Collab.	Access Control	Memory Provenance	MCP Support	Local Deploy
<i>Production Agents</i>																				
Claude Code	✓	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓
Gemini CLI	✓	⬢	⬢	✓	✓	✗	✗	⬢	✗	✗	⬢	⬢	✗	✗	✗	✗	✗	✗	✓	✓
Codex CLI	✓	✗	✗	✓	✓	✗	⬢	⬢	✗	✗	✗	⬢	✗	✗	✗	✗	✗	✗	✓	✓
Copilot	✓	✗	✗	✓	✓	✗	⬢	⬢	✗	✗	⬢	✗	✗	✗	✗	✗	✗	✗	✓	✗
Cursor	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗	⬢	✗	✗	✗	✗	✗	✗	✗	✓	✗
OpenCode	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓
Windsurf	✓	✗	✗	✓	⬢	✗	✗	✗	✗	✗	⬢	✗	✗	✗	✗	✗	✗	✗	✓	✗
Devin	⬢	✗	✗	⬢	⬢	✗	✗	✗	✗	✗	⬢	✗	✗	✗	✗	⬢	✗	✗	✓	✗
Aider	⬢	✗	✗	✗	✗	✗	✗	✗	✗	✗	⬢	✗	⬢	✗	✓	✗	✗	✗	⬢	✓
OpenHands	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓
<i>MCP Community Add-ons</i>																				
claude-mem	✓	✓	✗	✓	✓	✗	✗	⬢	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓	✓
Mem0	✓	⬢	✗	✓	✓	✗	✗	⬢	✗	✗	✗	✓	✗	✗	✗	⬢	✗	✗	✓	✓



03

Memory Architectures



Three Paradigms

Classification of ICLR 2026 MemAgents workshop contributions:

Extraction-Based

Mem0

Extract atomic facts
Vector-indexed retrieval

Excels at factual recall

Lossy by design

Self-Managing

Letta / MemGPT

LLM manages its own
memory

Hierarchical
read/write/search

Behavioral continuity

Less transparent

Graph-Based Temporal

Graphiti / Zep

Bi-temporal validity
intervals

Event time vs. ingestion
time

Causal reasoning

Added complexity

Benchmarked head-to-head on LoCoMo-Plus cognitive memory evaluation (Gurawaliya et al., 2026)



The Case for Hybrid Architectures

- LoCoMo-Plus results: **no single paradigm masters all competency types**
- The three paradigms capture genuinely different aspects of memory
- Emerging directions:
 - RL-based management (Memory-R1, AgeMem) — learned policies discover non-obvious strategies, e.g. preemptive summarization
 - Multi-graph memory (MAGMA) — semantic, temporal, causal, entity graphs queried independently
 - Hierarchical consolidation (TiMem) — temporal memory tree for long execution traces
 - Hybrid local/cloud (LightMem, HyMem) — small models route, large models consolidate



04

SE-Specific Mechanisms



Versioned Context & Decision Provenance

Intra-session:

- GCC — Git-inspired operations for agent working memory
- Branching, merging, rolling back context within a task

Inter-session:

- Lore — git commits as structured decision records
- Constraints, rejected alternatives, verification metadata via git trailers

Why it matters:

- Each deviation in a mutating action **reduces success odds**
- Errors grow as context length increases
- Agents drift from stale constraints: deprecated APIs, outdated conventions
- **Decision provenance** lets agents understand *why* before proposing changes



Experiential & Procedural Learning

- SWE Context Bench: summarized experience improves resolution; **unfiltered experience hurts**
- Experiential Co-Learning — mine experiences from historical trajectories
- ERL — reflect on trajectories → transferable heuristics; self-generated experiences outperform externally seeded ones
- Live-SWE-agent — self-creates new tools during execution (procedural memory via self-modification)
- Mem^p — distills trajectories into step-by-step instructions + script-like abstractions

Capacity Threshold

7B models do not benefit from experiential memory. 32B+ models show pronounced gains. (WebCoach, 2026)



Knowledge Graphs & Collaboration

Graph memory for SE:

- **Dependency tracking** — files, imports, class hierarchies
- **Decision provenance** — ADRs, review discussions, commit rationale
- **Temporal evolution** — API validity, convention adoption, deprecation

GAM: hierarchical graph memory significantly outperforms extraction-based approaches

Collaborative memory:

- Strong single-agent bias in current research
- Private memory (debugging notes) vs. shared memory (conventions, decisions)
- Security: MINJA shows shared-memory agents vulnerable to **injection attacks**
- No SE-specific governance: repo permissions, review approvals, release cycles



05

Open Gaps & Vision



Six Open Gaps

1. Cross-session learning — resolution trajectories discarded after each session; no production harness integrates experiential transfer
2. Temporal knowledge validity — no SE-specific temporal memory; only one SE-specific temporal benchmark exists
3. Procedural memory — SE craft knowledge is overwhelmingly procedural, yet it is the rarest type in research and production
4. Collaborative memory — teams need shared decisions, private notes, and repository-aligned access control
5. Governance & security — memory injection attacks, GDPR-compliant deletion entirely unaddressed
6. MCP orchestration — no layer decides which memory to consult when, resolves conflicts, or manages lifecycle across layers



Key Takeaway

The Central Argument

Persistent, temporally grounded memory is the prerequisite for software engineering agents that learn alongside the teams they serve.

- The factual-to-cognitive gap across production architectures is confirmed by benchmarks
- The MCP ecosystem provides composable building blocks — but assembling the full stack remains a manual integration challenge
- Closing these gaps requires transforming SE agents from **amnesic tools** into **learning collaborators**

Banf, Kuhn, Garcarz, Becker — Perelyn GmbH, Munich



Discussion

Questions & Discussion

- How do you handle memory in your SE agent workflows today?
- Which memory type (episodic, procedural, temporal) would have the highest impact for your use case?
- Should memory governance be agent-managed or developer-controlled?

michael.banf@perelyn.com



MEMORY-AWARE SOFTWARE ENGINEER- ING AGENTS

AI4SE @ KI 2026

Architectures, Mechanisms, and Open Gaps